

평균 예측

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 개요
2. 평균의 성질
3. 변동성 (Variability)
4. SD와 정규곡선 (SD and Normal Curve)
5. 중심극한정리
6. 표본평균의 변동성
7. 표본크기 결정

개요

평균 예측

- 평균 예측 개요

- 표본평균의 경험적 분포는 모집단의 분포와 무관하게 항상 종 모양(bell-shaped)

- 핵심 포인트들

- 평균은 정확히 무엇을 측정하는가?
 - 대부분의 데이터는 평균에서 얼마나 떨어져 있는가?
 - 표본 크기는 표본평균의 변동성과 어떤 관계인가?
 - 왜 랜덤 표본평균의 경험적 분포는 종 모양이 되는가?
 - 표본평균을 추론에 어떻게 효과적으로 사용할 수 있는가?

평균의 성질

평균의 성질 (1)

■ 평균 (Mean or Average)

- 평균 = 모든 원소의 합 / 원소의 개수

```
not_symmetric = make_array(2, 3, 3, 9)
```

```
np.average(not_symmetric)
```

```
np.mean(not_symmetric)
```

4.25

4.25

■ 평균의 기본 성질들

- 컬렉션의 원소일 필요가 없음
- 원소가 모두 정수여도 반드시 정수는 아님
- 최솟값과 최댓값 사이에 위치함
- 두 극단의 중간점 일 필요가 없다 → 절반의 원소가 평균 이상이라는 보장 없음
- 원래 데이터와 동일한 단위를 가짐

평균의 성질 (2)

■ 평균과 히스토그램

- 평균은 컬렉션의 원소 수가 아닌 **분포(값의 비율)**에만 의존 → 같은 분포를 가진 두 컬렉션은 반드시 같은 평균을 가짐
- 평균 = 히스토그램의 무게중심 (center of gravity) = 지렛점 (fulcrum)

```
mean = 4.25
```

$$= \frac{2 + 3 + 3 + 9}{4}$$

$$= 2 \cdot \frac{1}{4} + 3 \cdot \frac{1}{4} + 3 \cdot \frac{1}{4} + 9 \cdot \frac{1}{4}$$

$$= 2 \cdot \frac{1}{4} + 3 \cdot \frac{2}{4} + 9 \cdot \frac{1}{4}$$

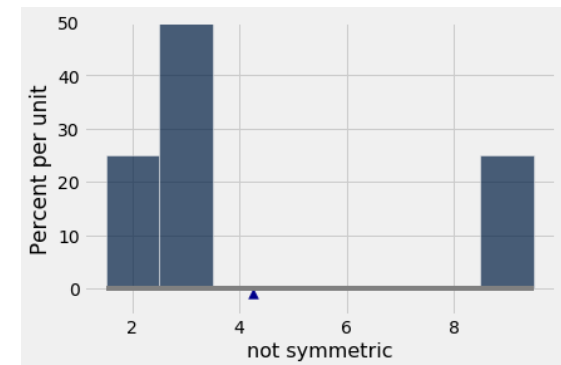
$$= 2 \cdot 0.25 + 3 \cdot 0.5 + 9 \cdot 0.25$$

```
not_symmetric
```

```
array([2, 3, 3, 9])
```

```
same_distribution = make_array(2, 2, 3, 3, 3, 3, 9, 9)  
np.mean(same_distribution)
```

4.25



평균: 4.25

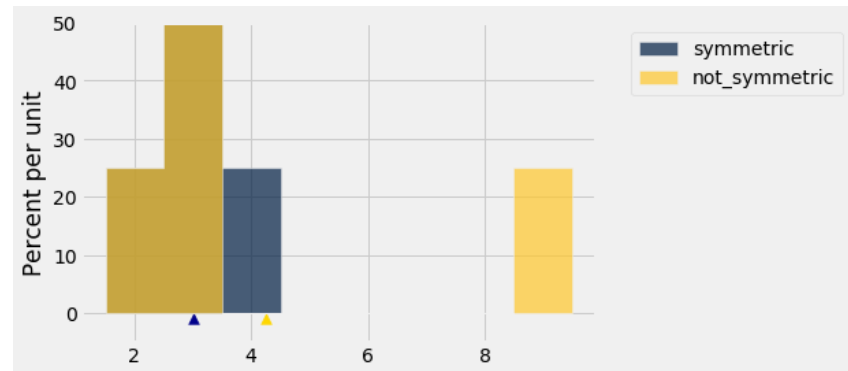
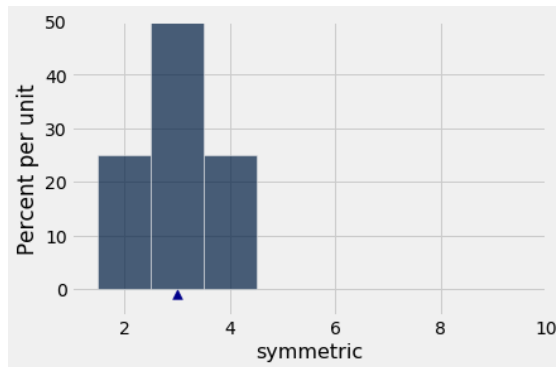
평균의 성질 (3)

■ 평균과 중앙값(Median)

■ 왜도 (Skewness)의 영향

- 히스토그램이 한쪽으로 치우친 경우, 평균은 중앙값에서 꼬리 방향으로 끌림

	대칭분포	우편향 (Right-skewed)
예시	[2, 3, 3, 4]	[2, 3, 3, 9]
평균	3	4.25
중앙값	3	3
형태	히스토그램이 평균에 대해 대칭	평균이 꼬리 방향으로 당겨짐



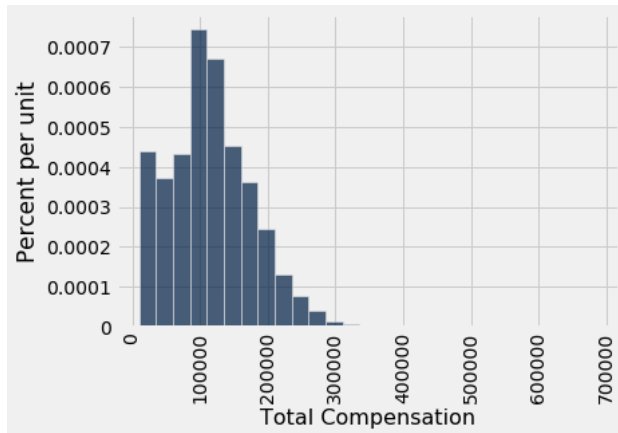
평균의 성질 (4)

- 예제: 샌프란시스코 2015 공무원 급여

- https://github.com/data-8/materials-sp18/blob/master/lec/san_francisco_2015.csv

```
sf2015 = Table.read_table(path_data + 'san_francisco_2015.csv').where('Salaries', are.above(10000))
```

```
sf2015.select('Total Compensation').hist(bins = np.arange(10000, 700000, 25000))
```



```
compensation = sf2015.column('Total Compensation')  
percentile(50, compensation)
```

110305.79

```
np.mean(compensation)
```

114725.98411824222

변동성

변동성 (1)

■ 표준편차 (Standard deviation) 계산

- ① 평균 계산
- ② 편차 (deviation) 계산: 각 값 - 평균
- ③ 편차 (squared deviation)를 제곱
- ④ 분산 (variance) = 제곱 편차의 평균
- ⑤ 표준편차(SD) = $\sqrt{\text{분산}}$

```
any_numbers = make_array(1, 2, 2, 10)
```

```
# Step 1. The average.
```

```
mean = np.mean(any_numbers)  
mean
```

3.75

```
# Step 2. The deviations from average.
```

```
deviations = any_numbers - mean  
calculation_steps = Table().with_columns(  
    'Value', any_numbers,  
    'Deviation from Average', deviations  
)  
calculation_steps
```

Value	Deviation from Average
1	-2.75
2	-1.75
2	-1.75
10	6.25

```
# Step 3. The squared deviations from average
```

```
squared_deviations = deviations ** 2  
calculation_steps = calculation_steps.with_column(  
    'Squared Deviations from Average', squared_deviations  
)  
calculation_steps
```

Value	Deviation from Average	Squared Deviations from Average
1	-2.75	7.5625
2	-1.75	3.0625
2	-1.75	3.0625
10	6.25	39.0625

```
# Step 4. Variance = the mean squared deviation from average
```

```
variance = np.mean(squared_deviations)  
variance
```

13.1875

```
# Step 5.
```

```
# Standard Deviation:    root mean squared deviation from average  
# Steps of calculation:  5    4    3    2    1
```

```
sd = variance ** 0.5  
sd
```

3.6314597615834874

```
np.std(any_numbers)
```

3.6314597615834874

표준편차
계산함수

변동성 (2)

■ 예시: NBA 선수 데이터

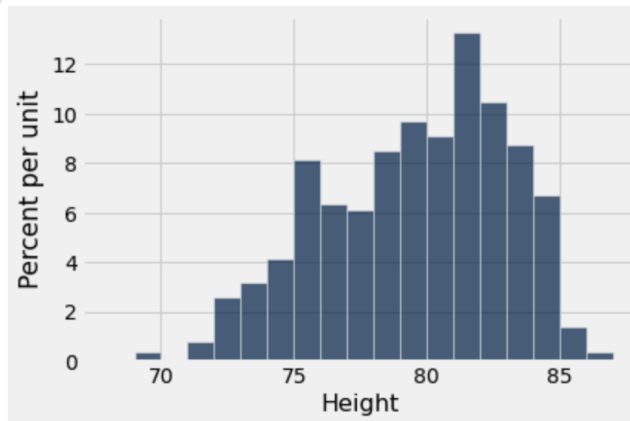
- <https://github.com/data-8/materials-sp18/blob/master/lec/nba2013.csv>

```
nba13 = Table.read_table(path_data + 'nba2013.csv')
nba13
```

Name	Position	Height	Weight	Age in 2013
DeQuan Jones	Guard	80	221	23
Darius Miller	Guard	80	235	23
Trevor Ariza	Guard	80	210	28
James Jones	Guard	80	215	32
Wesley Johnson	Guard	79	215	26
Klay Thompson	Guard	79	205	23
Thabo Sefolosha	Guard	79	215	29
Chase Budinger	Guard	79	218	25
Kevin Martin	Guard	79	185	30
Evan Fournier	Guard	79	206	20

... (495 rows omitted)

```
nba13.select('Height').hist(bins=np.arange(68, 88, 1))
```



```
mean_height = np.mean(nba13.column('Height'))
mean_height
```

79.06534653465347

```
sd_height = np.std(nba13.column('Height'))
sd_height
```

3.4505971830275546

```
nba13.sort('Height', descending=True).show(3)
```

Name	Position	Height	Weight	Age in 2013
Hasheem Thabeet	Center	87	263	26
Roy Hibbert	Center	86	278	26
Alex Len	Center	85	255	20

... (502 rows omitted)

```
87 - mean_height
```

7.934653465346528

```
(87 - mean_height)/sd_height
```

2.2995015194397923

```
nba13.sort('Height').show(3)
```

Name	Position	Height	Weight	Age in 2013
Isaiah Thomas	Guard	69	185	24
Nate Robinson	Guard	69	180	29
John Lucas III	Guard	71	157	30

... (502 rows omitted)

```
(69 - mean_height)/sd_height
```

-2.9169868288775844

가장 키 크고 가장 작은 선수도 평균에서 불과 3 SD 이내

➔ SD는 데이터가 얼마나 흩어져 있는지를 요약하는 유용한 척도

변동성 (3)

■ 체비쇼프 부등식 (Chebychev's Bounds)

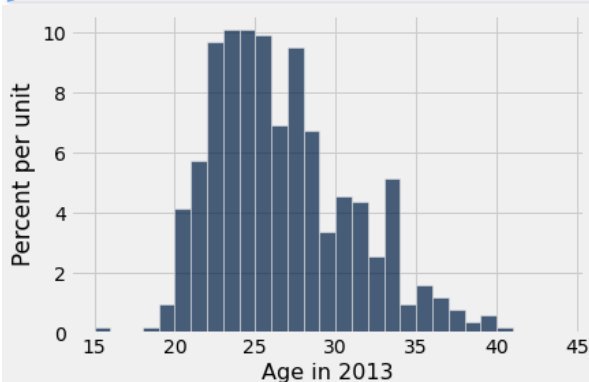
- 체비쇼프는 하한(lower bound)을 제공할 뿐, 정확한 값이나 근사치가 아니다. 하지만 어떤 분포에도 보편적으로 적용

모든 수치 데이터 리스트에 대해, "평균 $\pm z$ SD" 범위에 속하는 원소의 비율은 최소 $1 - 1/z^2$ ($z > 1$)

- the proportion in the range "average ± 2 SDs" is **at least** $1 - 1/4 = 0.75$
- the proportion in the range "average ± 3 SDs" is **at least** $1 - 1/9 \approx 0.89$
- the proportion in the range "average ± 4.5 SDs" is **at least** $1 - 1/4.5^2 \approx 0.95$

■ 예시

```
nba13.select('Age in 2013').hist(bins=np.arange(15, 45, 1))
```



```
ages = nba13.column('Age in 2013')
mean_age = np.mean(ages)
sd_age = np.std(ages)
mean_age, sd_age
```

```
(26.19009900990099, 4.321200441720307)
```

```
nba13.sort('Age in 2013', descending=True).show(3)
```

Name	Position	Height	Weight	Age in 2013
Juwan Howard	Forward	81	250	40
Steve Nash	Guard	75	178	39
Derek Fisher	Guard	73	210	39
... (502 rows omitted)				

```
(40 - mean_age)/sd_age
```

```
3.1958482778922357
```

변동성 (4)

■ 표준 단위

- 표준 단위

$$z = \frac{\text{value} - \text{average}}{\text{SD}}$$

- 체비쇼프 부등식에 의해, 표준 단위 z값은 대부분 (-5, 5) 범위에 머뭇

■ 예시: United airlines 결항 지연 데이터

- https://github.com/data-8/materials-sp18/blob/master/lec/united_summer2015.csv

```
united = Table.read_table(path_data + 'united_summer2015.csv')
united = united.with_column(
    'Delay (Standard Units)', standard_units(united.column('Delay'))
)
united
```

Date	Flight Number	Destination	Delay	Delay (Standard Units)
6/1/15	73	HNL	257	6.08766
6/1/15	217	EWR	28	0.287279
6/1/15	237	STL	-3	-0.497924
6/1/15	250	SAN	0	-0.421937

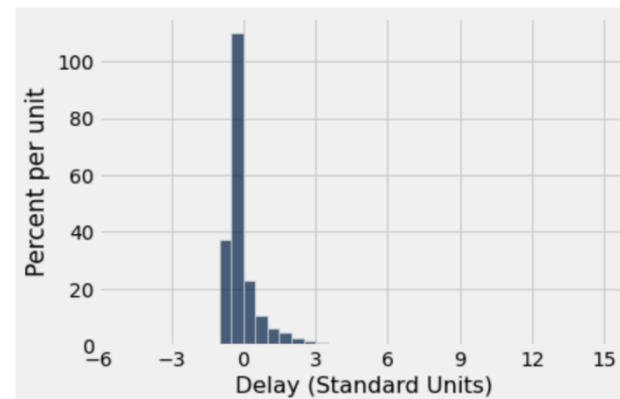
```
united.sort('Delay', descending=True)
```

Date	Flight Number	Destination	Delay	Delay (Standard Units)
6/21/15	1964	SEA	580	14.269
6/22/15	300	HNL	537	13.1798
6/21/15	1149	IAD	508	12.4453
6/20/15	353	ORD	505	12.3693

```
within_3_sd = united.where('Delay (Standard Units)', are.between(-3, 3))
within_3_sd.num_rows/united.num_rows
```

0.9790235081374322

```
united.hist('Delay (Standard Units)', bins=np.arange(-5, 15.5, 0.5))
plots.xticks(np.arange(-6, 17, 3));
```



SD와 정규곡선

SD와 정규곡선 (1)

- 종 모양(Bell-shaped) 분포에서의 SD 찾기
 - <https://github.com/data-8/materials-sp18/blob/master/lec/baby.csv>
 - 산모 신장 데이터 (1,174명) : 평균 64인치, 표준편차 2.5인치
 - 종 모양 분포에서 SD = 평균과 변곡점(inflection point) 사이의 거리
 - 변곡점: 곡선이 "위로 오목(upside-down cup)"에서 "아래로 오목(right-way-up cup)"으로 바뀌는 지점

```
baby = Table.read_table(path_data + 'baby.csv')
```

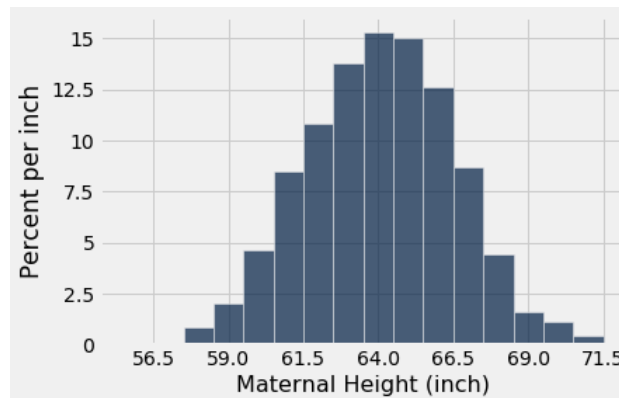
```
heights = baby.column('Maternal Height')  
mean_height = np.round(np.mean(heights), 1)  
mean_height
```

64.0

```
sd_height = np.round(np.std(heights), 1)  
sd_height
```

2.5

```
baby.hist('Maternal Height', bins=np.arange(55.5, 72.5, 1), unit='inch')  
positions = np.arange(-3, 3.1, 1)*sd_height + mean_height  
plots.xticks(positions);
```



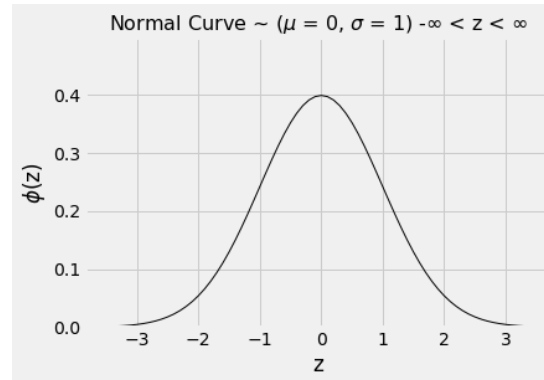
즉, $z = \pm 1$ 지점이 변곡점 $\rightarrow$ 평균 ± 1 SD = $64 \pm 2.5 = \{61.5, 66.5\}$ 인치 16

SD와 정규곡선 (2)

- 표준 정규 곡선 (Standard normal curve)

- 모든 종 모양 히스토그램은 축의 레이블만 다를 뿐 본질적으로 동일한 하나의 곡선. 표준 단위로 변환했을 때의 이 기본 곡선을 표준 정규곡선이라 함

$$\phi(z) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}, \quad -\infty < z < \infty$$



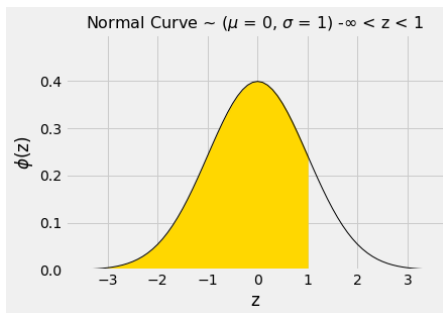
- 표준 정규곡선 주요 성질

- 곡선 아래 전체 넓이 = 1 (밀도 척도 히스토그램으로 해석)
- $z = 0$ 에 대해 대칭 → 평균 = 중앙값 = 0
- 변곡점 위치: $z = -1$ 과 $z = +1$
- SD = 1 (히스토그램에서 SD가 명확히 보이는 희귀한 분포)

SD와 정규곡선 (3)

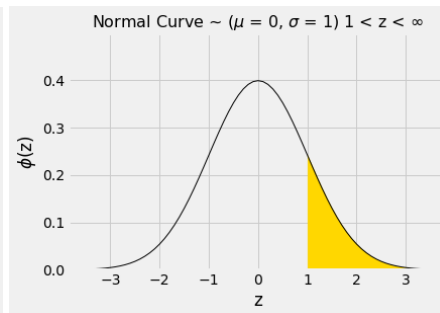
표준 정규 CDF

- 정규곡선은 일반적인 미적분으로 적분이 불가능하여, Python의 `scipy.stats.norm.cdf`를 사용



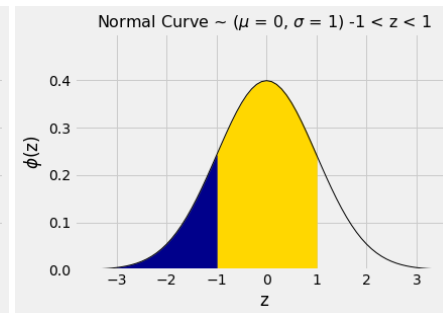
```
stats.norm.cdf(1)
```

0.8413447460685429



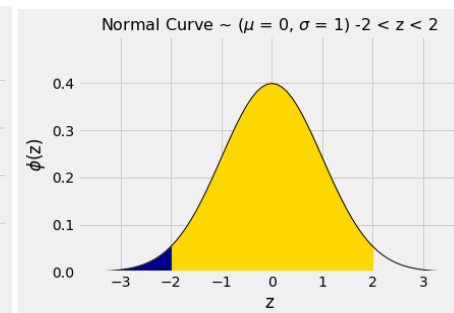
```
1 - stats.norm.cdf(1)
```

0.15865525393145707



```
stats.norm.cdf(1) - stats.norm.cdf(-1)
```

0.6826894921370859



```
stats.norm.cdf(2) - stats.norm.cdf(-2)
```

0.9544997361036416

Percent in Range	All Distributions: Bound	Normal Distribution: Approximation
average ± 1 SD	at least 0%	about 68%
average ± 2 SDs	at least 75%	about 95%
average ± 3 SDs	at least 88.888...%	about 99.73%

분포가 정규분포임을 알면 하한 대신 정확한 근사치를 사용할 수 있어 훨씬 강력

중심 극한 정리

중심 극한 정리 (1)

■ 예시1: 룰렛 순이익

- 이항분포임에도 불구하고, 큰 표본의 합 또는 평균의 분포는 종 모양이 됨

wheel

```
red = wheel.with_column(  
    'Winnings: Red', wheel.apply(red_winnings, 'Color')  
)  
red
```

Pocket	Color	Winnings: Red
0	green	-1
00	green	-1
1	red	1
2	black	-1
3	red	1
4	black	-1
5	red	1
6	black	-1
7	red	1

```
def red_winnings(color):  
    if color == 'red':  
        return 1  
    else:  
        return -1
```

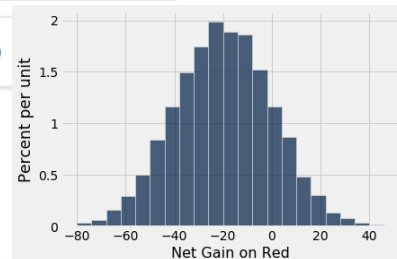
```
num_bets = 400  
repetitions = 10000
```

```
net_gain_red = make_array()
```

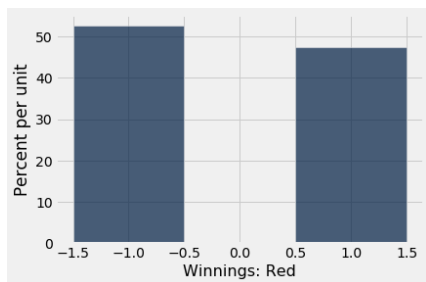
```
for i in np.arange(repetitions):  
    spins = red.sample(num_bets)  
    new_net_gain_red = spins.column('Winnings: Red').sum()  
    net_gain_red = np.append(net_gain_red, new_net_gain_red)
```

```
results = Table().with_column(  
    'Net Gain on Red', net_gain_red  
)
```

```
results.hist(bins=np.arange(-80, 50, 6))
```



```
red.select('Winnings: Red').hist(bins=np.arange(-1.5, 1.6, 1))
```



```
average_per_bet = 1*(18/38) + (-1)*(20/38)  
average_per_bet
```

-0.05263157894736842

```
400 * average_per_bet
```

-21.052631578947366

```
np.mean(results.column(0))
```

-20.9586

```
np.std(results.column(0))
```

20.029115957525438

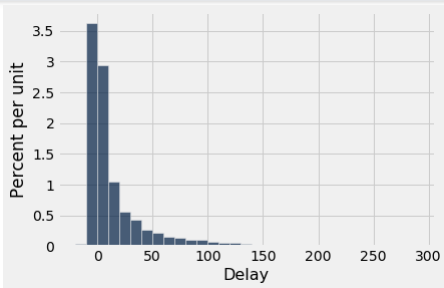
중심 극한 정리 (2)

■ 예시2: 비행 지연 평균

- 13,825 국내 비행 지연
- 분포가 우편향임에도 불구하고, 큰 표본의 합 또는 평균의 분포는 종 모양

```
united = Table.read_table(path_data + 'united_summer2015.csv')
```

```
united.select('Delay').hist(bins=np.arange(-20, 300, 10))
```



```
mean_delay = np.mean(united.column('Delay'))  
sd_delay = np.std(united.column('Delay'))
```

```
mean_delay, sd_delay
```

```
(16.658155515370705, 39.480199851609314)
```

```
delay = united.select('Delay')
```

```
np.mean(delay.sample(400).column('Delay'))
```

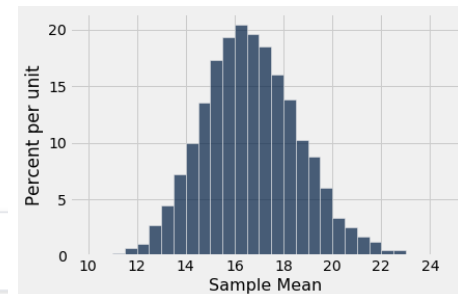
```
sample_size = 400  
repetitions = 10000
```

```
means = make_array()
```

```
for i in np.arange(repetitions):  
    sample = delay.sample(sample_size)  
    new_mean = np.mean(sample.column('Delay'))  
    means = np.append(means, new_mean)
```

```
results = Table().with_column(  
    'Sample Mean', means  
)
```

```
results.hist(bins=np.arange(10, 25, 0.5))
```



중심 극한 정리 (3)

■ 중심 극한 정리 (Central limit theorem)

(정의) 대형 랜덤 표본(복원 추출)의 합 또는 평균의 확률 분포는, 모집단의 분포에 무관하게, 근사적으로 정규분포를 따름

- 모집단 분포를 거의 모르는 상황에서도 대형 랜덤 표본만 있으면 추론이 가능

■ 예시: 멘델 완두 꽃

```
colors = make_array('Purple', 'Purple', 'Purple', 'White')

model = Table().with_column('Color', colors)

model
```

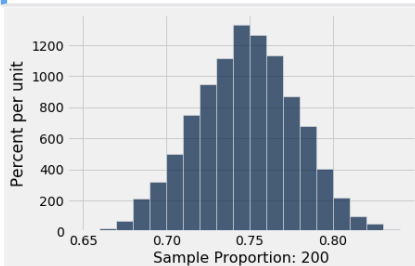
```
props = make_array()

num_plants = 200
repetitions = 10000

for i in np.arange(repetitions):
    sample = model.sample(num_plants)
    new_prop = np.count_nonzero(sample.column('Color') == 'Purple')/num_plants
    props = np.append(props, new_prop)

results = Table().with_column('Sample Proportion: 200', props)
```

```
results.hist(bins=np.arange(0.65, 0.85, 0.01))
```



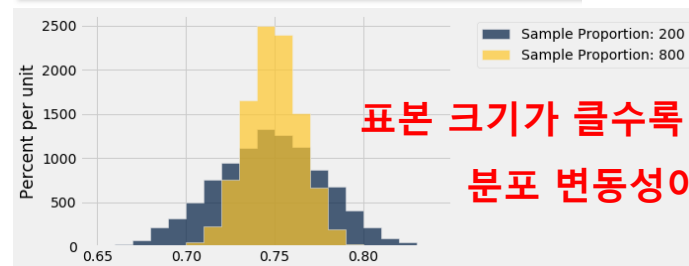
```
props2 = make_array()

num_plants = 800

for i in np.arange(repetitions):
    sample = model.sample(num_plants)
    new_prop = np.count_nonzero(sample.column('Color') == 'Purple')/num_plants
    props2 = np.append(props2, new_prop)

results = results.with_column('Sample Proportion: 800', props2)
```

```
results.hist(bins=np.arange(0.65, 0.85, 0.01))
```



표본 크기가 클수록 표본 비율의
분포 변동성이 감소

표본 평균의 변동성

표본 평균의 변동성 (1)

■ 표본평균도 확률변수

■ 모집단 평균 μ 추정

- 모집단 전체를 조사하는 것은 현실적으로 불가능
- 대안: 랜덤 표본을 추출하여 표본평균 $\bar{x}$ 계산 $\rightarrow \mu$ 의 추정 값으로 사용

■ $\bar{x}$ 는 확률변수

- 표본을 새로 뽑을 때마다 선택되는 원소가 달라짐. 따라서 $\bar{x}$ 도 매번 다른 값을 가짐
- $\bar{x}$ 는 고정된 숫자가 아니라 확률적으로 변하는 양 $\rightarrow$ 확률변수

표본평균은 모집단 평균의 불편 추정량(unbiased estimate)
 $\rightarrow$ 모든 가능한 표본에 대한 평균들의 평균 = 모집단 평균

핵심질문

$\bar{x}$ 는 어떤 분포를 따르는가?

중심극한정리(CLT)를 따름

표본이 충분히 크면, 모집단 분포 형태와 무관하게,
 $\bar{x}$ 의 분포 $\approx$ 정규분포 (종 모양) 의 분포 $\approx$ 정규분포 (종 모양)

종 모양의 중심 = 모집단 평균 μ

$\bar{x}$ 는 μ 주변에서 얼마나 퍼져 있는가?

표본 크기가 달라지면 종 모양의 너비가 달라짐

이 변동성을 정확히 수치로 표현하는 것이 필요

표본 크기 n 과 변동성은 어떤 관계인가?

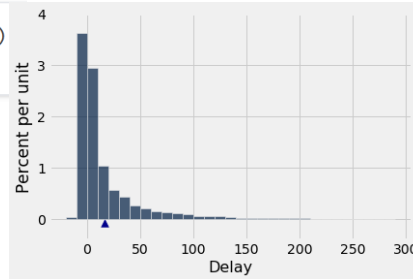
표본 평균의 변동성 (2)

■ 예시1: 비행지연

```
united = Table.read_table(path_data + 'united_summer2015.csv')
delay = united.select('Delay')
```

```
pop_mean = np.mean(delay.column('Delay'))
pop_mean
```

16.658155515370705



"""Empirical distribution of random sample means"""

```
def simulate_sample_mean(table, label, sample_size, repetitions):
```

```
    means = make_array()
```

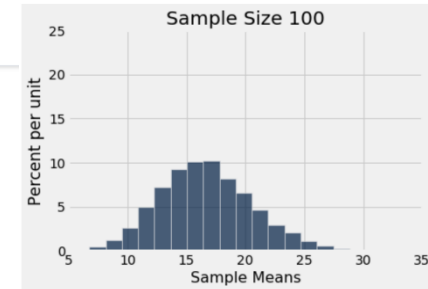
```
    for i in range(repetitions):
        new_sample = table.sample(sample_size)
        new_sample_mean = np.mean(new_sample.column(label))
        means = np.append(means, new_sample_mean)
```

```
    sample_means = Table().with_column('Sample Means', means)
```

```
    # Display empirical histogram and print all relevant quantities
    sample_means.hist(bins=20)
    plots.xlabel('Sample Means')
    plots.title('Sample Size ' + str(sample_size))
    print("Sample size: ", sample_size)
    print("Population mean:", np.mean(table.column(label)))
    print("Average of sample means: ", np.mean(means))
    print("Population SD:", np.std(table.column(label)))
    print("SD of sample means:", np.std(means))
```

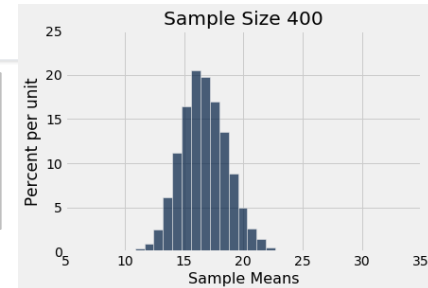
```
simulate_sample_mean(delay, 'Delay', 100, 10000)
plots.xlim(5, 35)
plots.ylim(0, 0.25);
```

Sample size: 100
Population mean: 16.658155515370705
Average of sample means: 16.672836
Population SD: 39.480199851609314
SD of sample means: 3.92467202924066



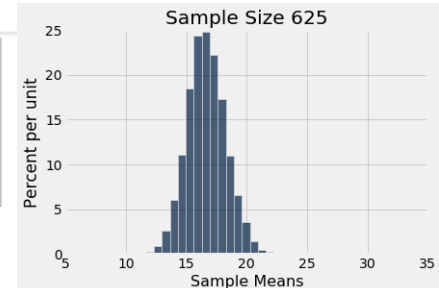
```
simulate_sample_mean(delay, 'Delay', 400, 10000)
plots.xlim(5, 35)
plots.ylim(0, 0.25);
```

Sample size: 400
Population mean: 16.658155515370705
Average of sample means: 16.678091499999997
Population SD: 39.480199851609314
SD of sample means: 1.9474592014668113



```
simulate_sample_mean(delay, 'Delay', 625, 10000)
plots.xlim(5, 35)
plots.ylim(0, 0.25);
```

Sample size: 625
Population mean: 16.658155515370705
Average of sample means: 16.649224
Population SD: 39.480199851609314
SD of sample means: 1.5883338034053167



표본 평균의 변동성 (3)

모든 표본 평균의 표준편차(SD)

모든 표본평균의 SD = 모집단 SD / $\sqrt{\text{표본크기}}$

- ① 비행 지연 데이터(n=13,825): 모집단 SD ≈ 39.48 분
- ② n=100 표본 10,000번 시뮬 → 표본평균들의 SD $\approx 3.92 \approx 39.48/\sqrt{100} = 3.95$
- ③ n=400 → SD $\approx 1.95 \approx 39.48/\sqrt{400} = 1.97$
- ④ n=625 → SD $\approx 1.59 \approx 39.48/\sqrt{625} = 1.58$
- ⑤ 시뮬레이션값과 pop_sd/ $\sqrt{n}$ 그래프가 거의 완벽히 일치 → 공식 확인

```
pop_sd = np.std(delay.column('Delay'))  
pop_sd
```

39.480199851609314

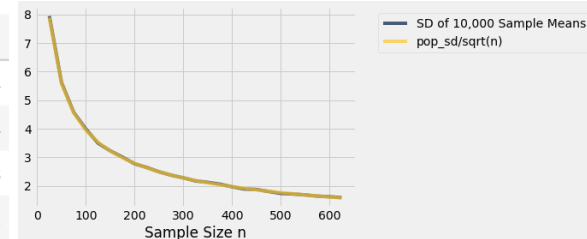
```
repetitions = 10000  
sample_sizes = np.arange(25, 626, 25)  
  
sd_means = make_array()  
  
for n in sample_sizes:  
    means = make_array()  
    for i in np.arange(repetitions):  
        means = np.append(means, np.mean(delay.sample(n).column('Delay'))  
    sd_means = np.append(sd_means, np.std(means))  
  
sd_comparison = Table().with_columns(  
    'Sample Size n', sample_sizes,  
    'SD of 10,000 Sample Means', sd_means,  
    'pop_sd/sqrt(n)', pop_sd/np.sqrt(sample_sizes)  
)
```

sd_comparison

Sample Size n	SD of 10,000 Sample Means	pop_sd/sqrt(n)
25	7.94482	7.89604
50	5.6131	5.58334
75	4.57417	4.55878
100	3.98687	3.94802
125	3.49769	3.53122
150	3.22776	3.22354
175	3.00675	2.98442
200	2.77764	2.79167
225	2.64268	2.63201
250	2.49447	2.49695

... (15 rows omitted)

sd_comparison.plot('Sample Size n')



표본 평균의 변동성 (4)

■ 정리

- 모집단 크기 무관: 공식에 모집단 크기 N 이 없다 → 수십억 명이든 1만 명이든 같은 n 이면 같은 정확도
- 표본 크기 $\uparrow$ → 정확도 $\uparrow$: 분모에 $\sqrt{n} \rightarrow n$ 이 커질수록 SD(평균)이 감소 → 추정치가 모집단 평균에 가까워짐
- 모집단 SD는 상수: 모든 표본이 같은 모집단에서 나옴 → 모집단 SD는 n 을 바꿔도 변하지 않음

■ 제곱근 법칙

- 표본 크기를 k 배 늘리면 SD(표본평균)은 $1/\sqrt{k}$ 배로 줄어듦 → 정확도는 $\sqrt{k}$ 배 향상

■ 최종통합정리

- 대형 랜덤 표본(복원 추출)에서 표본평균의 확률 분포
 - 근사 정규분포
 - 중심 = 모집단 평균
 - 표준편차 = 모집단 SD / $\sqrt{n}$
- 위의 3가지 모두 모집단 분포의 형태와 무관하게 성립

표본 크기 결정

표본 크기 결정 (1)

- 개요

- 왜 표본 크기를 미리 결정해야 하는가?
 - 실제 조사에서는 표본을 수집하기 전에 크기를 결정 - 수집 후 "부족했다"는 것을 알면 늦는다
 - 표본이 클수록 정확도는 높지만 비용·시간이 증가 ➔ 원하는 정확도를 달성하는 **최솟값** 을 찾는 것이 목표
 - 14.5에서 구한 공식 $SD(\bar{x}) = \text{모집단SD} / \sqrt{n}$ 을 역으로 활용
 - "n이 주어졌을 때 정확도가 얼마인가?" ➔ "원하는 정확도를 위해 n이 얼마여야 하는가?"

표본 크기 결정 (2)

■ 예시: 여론조사 시나리오

- 후보 A 지지율 추정을 위한 여론조사. 아래 조건에서 필요한 최소 표본 크기를 구하기
 - (가정) 유권자 수가 매우 많아 복원·비복원 추출 결과가 사실상 동일
 - (추정방법) CLT 근사 정규분포를 활용한 95% 신뢰구간 사용
 - (정확도 목표) 95% CI의 전폭 $\leq 2\%$ (반폭 $\leq 1\%$)

■ 5단계 풀이 (95% CI 폭에서 역산하기)

- ① 95% Confidence Interval (CI) 폭 = $4 \times \text{SD}(\text{표본비율})$ (정규곡선 "평균 $\pm 2 \text{ SD}$ " = 95% 포함)
- ② $\text{SD}(\text{표본비율}) = \text{SD}(0-1 \text{ 모집단}) / \sqrt{n}$ (14.5 공식 그대로)
- ③ 부등식 설정: $4 \times (\text{모집단 SD} / \sqrt{n}) \leq 0.01$
- ④ $\sqrt{n} \geq 4 \times \text{모집단 SD} / 0.01 \rightarrow$ 하지만 모집단 SD를 모름
- ⑤ 0-1 모집단의 SD 상한(≤ 0.5) 활용 $\rightarrow$ 보수적 최소 표본 크기 계산

➔ 지지율 p 를 모르기 때문에 SD도 모름. 해결책은 어떤 p 에 대해서도 성립하는 SD 상한을 이용해야 함

표본 크기 결정 (3)

- 0-1 모집단의 SD → 왜 최댓값이 0.5인가?
 - 모집단의 표준편차(SD)를 모르는 상황 → 모집단 SD의 최댓값은 알 수 있음
 - 비율 p 인 0-1 모집단의 분산과 SD를 직접 계산
 - 모집단: 비율 p 만큼 1, 비율 $(1-p)$ 만큼 0으로 구성
 - 평균 = $1 \times p + 0 \times (1-p) = p$
 - 분산 = $(1-p)^2 \times p + (0-p)^2 \times (1-p) = p(1-p)[(1-p) + p] = p(1-p)$
 - SD = $\sqrt{p(1-p)}$ $\frac{d}{dp} [p(1-p)] = \frac{d}{dp} [p - p^2]$
 - SD를 최대화: $= 1-2p = 0 \rightarrow p = 0.5$ 에서 최대
 - 최댓값: SD = $\sqrt{(0.5 \times 0.5)} = 0.5$

비율 p	$1-p$	$p(1-p)$	SD = $\sqrt{p(1-p)}$	비고
0.1	0.9	0.09	0.300	한쪽으로 몰림
0.3	0.7	0.21	0.458	
0.5	0.5	0.25	0.500	
0.7	0.3	0.21	0.458	최댓값 ← SD 상한 사용
0.9	0.1	0.09	0.300	

$p=0.1$ 과 대칭

표본 크기 결정 (4)

0-1 모집단의 SD → 왜 최댓값이 0.5인가? (2)

- 2가지 경우 비교
 - 반반 섞여 있으면 → 가장 퍼져 있음 → SD 최대
 - 한쪽으로 몰려 있으면 → 덜 퍼짐 → SD 감소

	Case 1 (0과 1이 반반)	Case 2 (90%가 1)
데이터	[1 1 1 1 1 0 0 0 0 0]	[1 1 1 1 1 1 1 1 1 0]
평균	0.5	0.9
편차	1들은 평균보다 +0.5만큼 떨어져 있고 0들은 평균보다 -0.5만큼 떨어져 있음	- 대부분의 값이 평균 가까이에 몰려 있음 - 데이터가 덜 퍼져 있음
표준편차	SD=5	SD=0.3 (근사값)

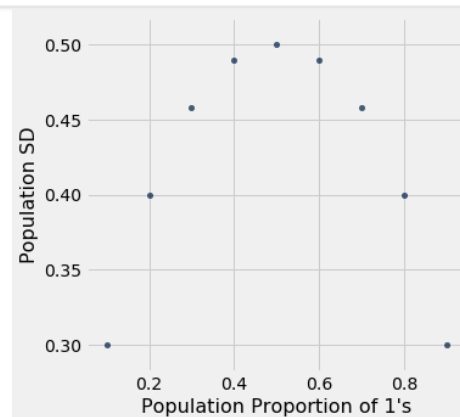
```
sd = make_array()
for i in np.arange(1, 10, 1):
    # Create an array of i 1's and (10-i) 0's
    population = np.append(np.ones(i), 1-np.ones(10-i))
    sd = np.append(sd, np.std(population))

zero_one_sds = Table().with_columns(
    "Population Proportion of 1's", np.arange(0.1, 1, 0.1),
    "Population SD", sd
)

zero_one_sds
```

Population Proportion of 1's	Population SD
0.1	0.3
0.2	0.4
0.3	0.458258
0.4	0.489898
0.5	0.5
0.6	0.489898
0.7	0.458258
0.8	0.4
0.9	0.3

```
zero_one_sds.scatter("Population Proportion of 1's")
```



표본 크기 결정 (5)

■ 최소 표본 크기 도출

① 조건: CI 전폭 $\leq 0.02 \rightarrow 4 \times \text{pop_SD} / \sqrt{n} \leq 0.02$

② pop_SD 상한 0.5 대입: $4 \times 0.5 / \sqrt{n} \leq 0.02$

③ 정리: $2 / \sqrt{n} \leq 0.02 \rightarrow \sqrt{n} \geq 100$

④ 양변 제곱: $n \geq 100^2 = 10,000$

➔ CI 전폭 2% 이하의 95% CI를 보장하려면 최소 10,000명 표본 필요

(CI 반폭 1% = 전폭 2%이므로 $n \geq 10,000$)

전제 조건	사용하는 pop_SD	필요 n (전폭 2%)	비고
p 완전히 모름 (보수적)	0.5 (상한)	10,000	항상 안전
p ≈ 0.3 or 0.7 예상	0.458	8,353	약 16% 절감
p ≈ 0.1 or 0.9 예상	0.300	3,600	약 64% 절감
p = 0.5 로 예상	0.500	10,000	보수적 값과 동일

➔ p 사전 정보가 있으면 더 작은 n으로도 충분 — 하지만 틀렸을 경우 목표 정확도를 보장할 수 없다. 안전을 원한다면 항상 0.5 상한을 사용

Q & A
